

Curso de Especialización

Ciberseguridad en las TIC

**Detección de problemas de
seguridad en apps móviles**

Objetivos

- Identificar los principales riesgos en aplicaciones móviles.
- Diferenciar entre ecosistemas Android e iOS desde la seguridad.
- Conocer OWASP Mobile Top 10.
- Saber las técnicas de detección de vulnerabilidades.
- Conocer las herramientas.
- Saber las buenas prácticas de seguridad para las aplicaciones móviles.

Resumen

Autora: Marina del Pilar López Oliva

Abril 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	28/04/2026	Documentación sobre la presentación.

Índice de Contenidos

1. Principales riesgos en aplicaciones móviles.....	4
2. Ecosistema Android vs iOS.....	5
3. OWASP Mobile Top 10.....	5
4. Técnicas de detección de vulnerabilidades.....	6
5. Herramientas utilizadas.....	7
6. Buenas prácticas de seguridad.....	7
7. Conclusiones.....	8

1. Principales riesgos en aplicaciones móviles.

Las aplicaciones móviles gestionan de forma constante información sensible, como credenciales, datos personales o tokens de sesión, lo que las convierte en un objetivo prioritario para los atacantes. A pesar de la evolución de las plataformas, existen vulnerabilidades que siguen apareciendo de forma recurrente en auditorías de seguridad.

Uno de los problemas más frecuentes es el almacenamiento inseguro de datos. En muchos casos, la información crítica se guarda en texto plano o con mecanismos de protección insuficientes. Por ejemplo, almacenar un token de autenticación sin cifrar en SharedPreferences o en una base de datos local permite que un atacante con acceso al dispositivo pueda extraerlo fácilmente.

Otro riesgo relevante son las comunicaciones inseguras. Aunque el uso de HTTPS está ampliamente extendido, aún se detectan aplicaciones que utilizan HTTP o que no validan correctamente los certificados TLS. Esto facilita ataques de tipo Man-in-the-Middle (MITM), en los que un atacante intercepta o modifica la comunicación entre la aplicación y el servidor.

El uso de librerías de terceros introduce también un vector de riesgo significativo. La integración de dependencias sin auditar puede derivar en la inclusión de código malicioso o vulnerabilidades conocidas dentro de la aplicación.

La autenticación débil sigue siendo una vulnerabilidad crítica. El uso de contraseñas poco robustas, la ausencia de autenticación multifactor (MFA) o la generación de tokens predecibles facilita accesos no autorizados.

Asimismo, las inyecciones de código continúan presentes en entornos móviles. Es posible encontrar casos de SQL Injection en bases de datos locales o XSS en WebViews mal configurados.

Por último, el exceso de permisos constituye un fallo habitual. Solicitar más permisos de los necesarios incrementa la superficie de ataque y supone un riesgo adicional para la privacidad del usuario.

Estos riesgos están alineados con los vectores recogidos en estándares como OWASP, lo que permite estructurar su análisis y mitigación de forma sistemática.

2. Ecosistema Android vs iOS.

El análisis de seguridad en aplicaciones móviles está estrechamente ligado al sistema operativo en el que se ejecutan. Android e iOS presentan modelos de seguridad diferenciados.

Android es un sistema de código abierto, lo que permite a la comunidad analizar su funcionamiento en profundidad. Sin embargo, esta apertura también facilita la identificación de vulnerabilidades por parte de actores maliciosos. Además, permite la instalación de aplicaciones desde fuentes externas (sideloading), lo que incrementa el riesgo de malware. Otro factor relevante es la fragmentación de versiones, que dificulta la aplicación homogénea de parches de seguridad.

A nivel técnico, Android implementa un sistema de aislamiento mediante sandbox basado en identificadores de usuario (UID) de Linux, junto con un modelo de permisos gestionado dinámicamente desde Android 6.0. No obstante, este modelo no es infalible y ha demostrado presentar vulnerabilidades en distintos escenarios.

Por otro lado, iOS adopta un enfoque más restrictivo. Apple controla tanto el hardware como el software, reduciendo la superficie de ataque. Las aplicaciones deben superar un proceso de revisión antes de su publicación en la App Store, y las actualizaciones del sistema se distribuyen de forma uniforme.

Además, iOS proporciona mecanismos de seguridad robustos como el Keychain, diseñado para el almacenamiento seguro de información sensible. A pesar de ello, el sistema no está exento de vulnerabilidades.

En conclusión, ningún sistema es completamente seguro; la seguridad final depende en gran medida de la correcta implementación de la aplicación.

3. OWASP Mobile Top 10.

El OWASP Mobile Top 10 (2024) constituye una referencia fundamental en el ámbito de la seguridad en aplicaciones móviles. Este listado recoge las vulnerabilidades más críticas identificadas en entornos reales.

Las principales categorías son:

- M1. Uso indebido de credenciales: gestión inadecuada de contraseñas y tokens.

- M2. Cadena de suministro insegura: uso de dependencias comprometidas.
- M3. Autenticación y autorización inseguras: fallos en el control de acceso.
- M4. Validación insuficiente de datos: entrada y salida no controlada.
- M5. Comunicaciones inseguras: falta de cifrado o validación TLS.
- M6. Problemas de privacidad: exposición de datos sensibles.
- M7. Protección binaria insuficiente: facilidad para ingeniería inversa.
- M8. Configuración incorrecta: malas prácticas de despliegue.
- M9. Almacenamiento inseguro: datos sensibles sin protección adecuada.
- M10. Criptografía insuficiente: uso incorrecto de algoritmos o implementaciones.

Este marco no solo permite clasificar vulnerabilidades, sino también establecer prioridades durante auditorías de seguridad. Una aplicación que mitiga estos riesgos cumple con un estándar ampliamente reconocido en la industria.

4. Técnicas de detección de vulnerabilidades.

La detección de vulnerabilidades en aplicaciones móviles requiere la combinación de diferentes enfoques complementarios.

Antes de aplicar estas técnicas, es recomendable definir un modelo de amenaza (threat model), que permita identificar los activos críticos, los posibles atacantes y los vectores de ataque.

El análisis estático (SAST) consiste en revisar el código fuente o el bytecode sin ejecutar la aplicación. Permite detectar problemas como credenciales embebidas, configuraciones inseguras o uso incorrecto de APIs.

El análisis dinámico (DAST) se centra en el comportamiento de la aplicación en ejecución. Esto permite identificar vulnerabilidades que solo aparecen en tiempo real, como fallos en la lógica de negocio o validaciones incorrectas.

El análisis de tráfico de red implica interceptar la comunicación entre la aplicación y el servidor mediante proxies. De esta forma se puede verificar si los datos viajan cifrados, si se validan correctamente los certificados o si es posible manipular las peticiones.

La ingeniería inversa permite analizar la aplicación desde su interior, descompilando archivos APK o IPA para estudiar su lógica, algoritmos y posibles vulnerabilidades ocultas.

En la práctica, la combinación de estas técnicas proporciona una cobertura mucho más completa que su uso individual.

5. Herramientas utilizadas.

En el análisis de seguridad móvil se emplea un conjunto de herramientas especializadas:

- MobSF (Mobile Security Framework): plataforma integral que combina análisis estático y dinámico. Es un framework que automatiza análisis estático y dinámico, generando informes detallados.
- Frida: herramienta avanzada para la instrumentación dinámica y modificación del comportamiento en tiempo real. Permite modificar el comportamiento de la app en tiempo real mediante hooking.
- Burp Suite: proxy de interceptación para analizar y manipular tráfico de red. Utilizado para analizar tráfico HTTP/HTTPS.
- Drozer: orientada a la evaluación de componentes expuestos en Android. Es una herramienta especializada en la evaluación de componentes expuestos en Android.
- jadx y apktool: utilizadas para la descompilación y análisis del código. Permiten descompilar aplicaciones para su análisis interno, es decir, visualizar el código Java desde una APK. Son para cuando quieres descuartizar el código para ver dónde residen los problemas.
- objection: interfaz simplificada basada en Frida, especialmente útil para pruebas rápidas. Simplifica el uso de Frida, facilitando pruebas rápidas. Es ideal para iOS y para las pruebas rápidas.

El conocimiento práctico de estas herramientas es fundamental para realizar auditorías efectivas. El uso combinado de varias herramientas permite realizar auditorías más completas y precisas.

6. Buenas prácticas de seguridad.

La seguridad debe integrarse desde las primeras fases del desarrollo para minimizar riesgos en producción.

El cifrado de datos en reposo es fundamental. Se deben emplear algoritmos robustos como AES-256 y utilizar los almacenes seguros del sistema (Keystore en Android y Keychain en iOS).

En cuanto a las comunicaciones, es imprescindible el uso de HTTPS con validación estricta de certificados. Además, el uso de certificate pinning añade una capa adicional de seguridad: esta técnica consiste en asociar la aplicación a un certificado o clave pública específica, de modo que si el servidor presenta un certificado diferente, la conexión se rechaza, mitigando ataques MITM incluso en redes comprometidas.

El principio de mínimo privilegio implica solicitar únicamente los permisos estrictamente necesarios para el funcionamiento de la aplicación.

La ofuscación del código contribuye a dificultar la ingeniería inversa, protegiendo la lógica interna de la aplicación.

Es fundamental implementar mecanismos de autenticación robustos, como MFA, tokens de corta duración y almacenamiento seguro de contraseñas mediante funciones de hash seguras.

También se recomienda detectar dispositivos comprometidos (root/jailbreak), ya que estos entornos reducen significativamente las garantías de seguridad del sistema.

Asimismo, es necesario realizar una auditoría continua de dependencias, dado que las librerías externas pueden introducir vulnerabilidades.

Por último, la seguridad debe integrarse en el pipeline mediante prácticas DevSecOps, incorporando análisis automatizados en CI/CD y una gestión adecuada de logs para evitar la exposición de información sensible.

7. Conclusiones.

La seguridad en aplicaciones móviles no debe considerarse una fase final, sino un proceso continuo integrado en todo el ciclo de desarrollo.

El uso de estándares como OWASP Mobile Top 10 permite identificar y priorizar riesgos de forma estructurada, mientras que la combinación de técnicas como SAST, DAST y pruebas manuales garantiza una detección más completa de vulnerabilidades.

La adopción de un enfoque DevSecOps permite incorporar la seguridad desde las primeras fases del desarrollo, reduciendo costes y mejorando la calidad del producto final.

En un contexto donde las aplicaciones móviles forman parte crítica de los sistemas de información, garantizar su seguridad no es opcional, sino un requisito fundamental para su correcta puesta en producción.